

Đánh giá MLOps PaaS theo kiến trúc MLOps tham chiếu

Hiện trạng, khoảng trống và lộ trình Degradation-Aware Continuous Training

Báo cáo kỹ thuật nội bộ

August 28, 2026

Abstract

Báo cáo đánh giá repository MLOps PaaS theo kiến trúc tham chiếu của Amou Najafabadi *et al.* [1], một systematic mapping study tổng hợp 93 nghiên cứu, 39 thành phần kiến trúc, 56 bước quy trình và các vai trò MLOps. Kết quả cho thấy repository đã có nền tảng tốt cho một MLOps PaaS: control plane đa tenant, registry version bất biến, training/execution backend, đóng gói image, serving gateway, thu thập production event và data-drift workflow. Tuy nhiên, hệ thống hiện gần với biến thể V1 (inference service) có retraining thủ công hơn là V3 (continuous training) khép kín: thiếu ground-truth feedback, data-quality/performance monitoring, diagnosis đa tín hiệu, policy quyết định action, evaluation gate và traffic splitting có đo lường. Báo cáo đề xuất một lộ trình nghiên cứu và hiện thực hoá “Degradation-Aware Adaptive Model Maintenance”, trong đó detection, diagnosis và action selection là ba lớp tách biệt; retraining chỉ là một action có điều kiện, không phải phản ứng mặc định.

1 Phạm vi và phương pháp đánh giá

1.1 Nguồn tham chiếu

Paper tham chiếu [1] mô tả MLOps bằng hai góc nhìn: *structure* (39 thành phần trong sáu nhóm: Data Curation, Storage & Versioning, ML Training, CI/CD, Inference, Infrastructure & Supporting Services) và *process* (56 bước quy trình tạo, triển khai, vận hành và bảo trì model). Công trình mô tả tám biến thể V1–V8: V1–V4 dùng model được huấn luyện từ đầu, còn V5–V8 tương ứng với V1–V4 nhưng dùng pretrained model. Bốn biến thể chính V1–V4 được phân biệt bởi cách serve model (Inference Service hoặc Inference Engine Pool) và cách retrain (manual hoặc continuous). Về stakeholder, paper không chỉ có sáu vai trò: framework nêu chín vai trò người dùng chính, năm chuyên biệt hoá vai trò và một automated stakeholder là System.

Paper được dùng như *khung tham chiếu*, không phải một checklist bắt buộc hay một chỉ dẫn thực thi. Một thành phần không xuất hiện trong repo không tự động là lỗi; nó là khoảng trống cần được cân nhắc theo mục tiêu sản phẩm và hướng nghiên cứu.

1.2 Bằng chứng repo và giới hạn

Đánh giá ưu tiên source code, database models, manifests và tests hơn README. Các luồng được kiểm tra gồm upload/build, training, inference, registry/deployment và drift. Các kết luận “thiếu” nghĩa là chưa thấy một contract hay implementation nền tảng trong source được kiểm tra, không có nghĩa là người dùng không thể tự đưa logic đó vào code training của mình.

2 Kết luận điều hành

Repository là một **nền tảng MLOps PaaS mạnh ở vận hành model**, nhưng chưa là một hệ thống **continuous training thích ứng đầu-cuối**. Điểm mạnh nằm ở control plane, versioned artifacts, execution backends, đóng gói/deployment và inference event ingestion. Điểm yếu chính nằm ở vòng phản hồi có nhãn, quality/performance monitoring và decision intelligence giữa alert với retraining.

Table 1: Đánh giá định hướng theo khung kiến trúc tham chiếu (heuristic).

Chiều cạnh	Mức	Nhận định
Nền tảng kiến trúc	7/10	Plane separation, state ownership và workflow nền tảng đã rõ.
Storage, registry, lineage model	7/10	Model version, image digest, artifact và training snapshot khá tốt; data/feature lineage còn thiếu.
Training và CI/CD	7/10	Có training runner, Docker/Argo/Kubeflow, packager và registry; chưa có CT policy/evaluation gate.
Inference và observability	6/10	Có gateway, worker version-specific, event stream, Prometheus và data drift; performance feedback chưa hoàn chỉnh.
Closed-loop lifecycle	4/10	Automatic drift tạo drift run, chưa tạo diagnosis hay retraining request.
Stakeholder/governance process	4/10	Có auth/tenant/audit nền tảng; chưa có approval workflow và role mapping cho maintenance.

Về biến thể kiến trúc (theo Fig. 6 của paper tham chiếu), implementation hiện gần nhất với **V1 – Inference Service + Manual Retraining**: ML Component Builder (model-packager) đóng gói model thành container image, ML Component Deployer (deployment task) đưa lên production, Inference Service (model-server + ml/dl-serving workers) phục vụ prediction. Retraining do người dùng submit training job thủ công qua control plane. Đây chưa phải V2 (Inference Engine Pool với dynamic model loading), và cũng chưa phải V3/V4 (Continuous Training pipeline tự động). Các biến thể V5–V8 dùng pretrained model, hiện nằm ngoài scope.

3 Đối chiếu cấu trúc kiến trúc — Mapping 39 thành phần

Phần này ánh xạ chi tiết từng thành phần trong 39 component của kiến trúc tham chiếu [1] sang implementation thực tế trong repository, phân theo sáu nhóm. Hình 1 tóm tắt quan hệ giữa sáu nhóm component và làm nổi bật decision loop chưa có trong implementation hiện tại.

3.1 Data Curation

Table 2: Ánh xạ nhóm Data Curation.

Component	Mức phủ	Bảng chứng trong repo	Khoảng trống
Data Collector	Một phần	Workspace assets cho code/data upload; training job nhận <code>data_snapshot_uri</code> .	Không có data ingestion pipeline hay automated collection từ external sources.
Data Source	Một phần	User upload data qua workspace; reference asset cho drift.	Platform không quản lý kết nối tới external data sources (APIs, IoT, databases).
Data Preprocessor	Thiếu	Preprocessing do user code trong training entry point.	Không có platform-managed preprocessing/validation pipeline.
Data Quality Checker	Thiếu	Chưa có implementation.	Thiếu hoàn toàn – cần cho CT pipeline để tránh retrain trên dữ liệu lỗi.

Component	Mức phủ	Bằng chứng trong repo	Khoảng trống
Data Labeler	Thiếu	Chưa có labeling workflow hay ground-truth annotation.	Thiếu – cần cho delayed-label feedback loop.

3.2 Storage & Versioning

Table 3: Ảnh xạ nhóm Storage & Versioning.

Component	Mức phủ	Bằng chứng trong repo	Khoảng trống
Data Catalog	Thiếu	Chưa có dataset catalog/metadata inventory.	Cần cho traceability data → model.
Dataset Repository	Một phần	S3 chứa training data snapshots (data_snapshot_uri); workspace assets.	Dataset không có entity/version riêng; latest-only, không versioned.
Raw Data Store	Một phần	S3 chứa raw/reference assets; production logs nằm trong PostgreSQL.	Chưa tách rõ raw, processed và curated data store.
Feature Store	Thiếu	Chưa có implementation.	Thiếu – features tính trực tiếp trong user code.
Code Repository	Có	code_snapshot_uri trong TrainingJob; Git repo cho platform code.	Đã đủ cho MVP.
Model Repository	Có	ModelVersion + ModelArtifact + ModelMetric trong registry app; immutable versions, metrics_summary, params_summary, artifacts (source, training_output, package, mlflow, metrics, params, model_insights, feature_importance, input_schema).	Rất tốt – đây là điểm mạnh.
Artifact Repository	Có	S3 + ModelArtifact entity lưu trữ packaged/container artifacts theo version; immutable URI/checksum.	Đã đủ cho V1; cần liên kết chặt hơn với candidate evaluation.
ML Metadata Repository	Có	TrainingJob tracking (JSON), TrainingOutput, TrainingJobEvent, RegistryEvent; MLflow integration.	Đã tốt; thiếu environment lineage và config lineage đầy đủ.

Component	Mức phủ	Bảng chứng trong repo	Khoảng trống
Feedback Database	Thiếu	PostgreSQL schema có confidence, latency_ms, status_code, request_id; chưa có feedback API/store cho ground truth.	Thiếu nghiêm trọng – event ID tồn tại nhưng chưa được client dùng để nối feedback; các trường runtime cũng chưa đi vào event.
Predictions Storage	Có	paas_production_logs table có event id và tenant/project/version scoping.	Có storage, nhưng event hiện thiếu confi- dence/latency/status/request correlation và ID chưa trả cho client.

3.3 ML Training

Table 4: Ảnh xạ nhóm ML Training.

Component	Mức phủ	Bảng chứng trong repo	Khoảng trống
Feature Engineering Pipeline	Một phần	Feature transformation hiện nằm trong user training/serving code; chưa có pipeline được platform quản lý.	Cần chuẩn hoá transform, version và lineage nếu feature logic ảnh hưởng CT.
ML Experiment Pipeline	Có	TrainingJob với entry point, code/data snapshot, MLflow integration; Docker local hoặc Argo/Kubeflow production; training-runner thực thi.	Experimentation metadata phụ thuộc user code/MLflow.
Continuous Training Pipeline	Thiếu	Chưa có CT pipeline tự động.	Đây là mục tiêu nghiên cứu chính.
Model Evaluator	Một phần	Metrics lưu qua MLflow/TrainingOutput; chưa có platform-level evaluation gate và chuẩn so sánh candidate/champion.	Cần EvaluationGate entity cho champion–challenger.
Fine-Tuning Module	Thiếu	Chưa có pre-trained model fine-tuning workflow.	Có thể bổ sung sau.

3.4 CI/CD

Table 5: Ảnh xạ nhóm CI/CD.

Component	Mức phủ	Bằng chứng trong repo	Khoảng trống
ML Component Builder	Có	Model-packager service build image từ training artifacts; Build model với status tracking, image_uri, image_digest, BuildInputAsset.	Rất tốt.
ML Component Deployer	Có	Deployment model, Endpoint entity, Kustomize/GitOps manifests; celery task orchestration.	Tốt.
ML Model Deployer	Thiếu	Chưa có dynamic model deployment vào một Inference Engine Pool; repo deploy containerized component.	Chỉ cần khi chuyển sang V2/V4.
ML Pipeline Builder	Thiếu	Chưa có – cần cho V3 (package ML pipeline code).	Cần khi chuyển sang CT variant.
ML Pipeline Deployer	Thiếu	Chưa có – cần cho V3 (deploy ML pipeline).	Cần khi chuyển sang CT variant.

3.5 Inference

Table 6: Ảnh xạ nhóm Inference.

Component	Mức phủ	Bằng chứng trong repo	Khoảng trống
Inference Service	Có	Model-server gateway + ML/DL serving workers; version-specific routing; authentication (JWT, API Key); Prometheus counters/histograms.	Tốt – đây là V1 implementation.
Inference Engine Pool	Thiếu	Không có dynamic model loading engine.	Không cần cho V1/V3 approach.
Runtime Model Monitor	Một phần	Prometheus prediction-count/latency metrics; Redpanda event stream; consumer persistence; Evidently data drift.	Thiếu: performance monitoring (F1/recall trên labels), prediction distribution drift, confidence tracking trong event stream.
Trigger	Một phần	Automatic drift outbox và trigger threshold tạo DriftRun khi đủ số bản ghi.	Chỉ trigger drift run, không trigger CT pipeline hay evaluation.
Model Comparison Runner	Thiếu	Frontend comparison trả về metrics diff rõ ràng và recommendation “unknown”.	Cần bổ sung – champion–challenger evaluation là core của CT.

Component	Mức phủ	Bằng chứng trong repo	Khoảng trống
Decision Processor	Thiếu	Chưa có.	Thay thế bằng Diagnosis + RetrainingRequest trong lộ trình.
XAI Module	Thiếu	Chưa có explainability module.	Nice-to-have, không phải MVP.

3.6 Infrastructure & Supporting Services

Table 7: Ảnh xạ nhóm Infrastructure & Supporting Services.

Component	Mức phủ	Bằng chứng trong repo	Khoảng trống
Resource Manager	Một phần	TrainingJob có vcpu, memory_mb, max_runtime_seconds, accelerator_type/count. K8s resource limits.	Thiếu cost observability và GPU/DCGM metrics.
Communication Middleware	Có	Redpanda (Kafka-compatible) cho production events; EventOutbox trong control plane; Celery + Redis cho task queue.	Rất tốt.
Container Orchestrator	Có	Docker Compose (local); Kubernetes/K3s (production); container images qua model-packager.	Đã đủ.
Workflow Orchestrator	Có	Celery cho async tasks; Argo Workflows/Kubeflow cho production training; GitOps promotion.	Tốt.
Log Manager	Có	paas_production_logs table; TrainingJobEvent; RegistryEvent; EventOutbox.	Đã đủ.
API Gateway	Có	Model-server FastAPI gateway; Traefik reverse proxy; control plane Django REST APIs.	Tốt.
MLOps User Interaction Manager	Một phần	React/Vite web frontend; monitoring dashboard; training/deployment management UI.	Cần hoàn thiện evidence visualization, approval inbox và comparison thật.

Table 8: Tổng hợp mức phủ 39 thành phần theo nhóm.

Nhóm	Có	Một phần	Thiếu	Nhận định
Data Curation (5)	0	2	3	Yếu nhất; cần Data Quality Checker và Feedback cho CT.
Storage & Versioning (10)	5	2	3	Model/Artifact Repository mạnh; thiếu Feedback Database nghiêm trọng.
ML Training (5)	1	2	2	Experiment Pipeline tốt; CT Pipeline là mục tiêu nghiên cứu.
CI/CD (5)	2	0	3	Component build/deploy có cho V1; thiếu model/pipeline deployer theo các variant khác.
Inference (7)	1	2	4	Serving tốt; thiếu Model Comparison Runner và performance monitoring.
Infrastructure & Supporting Services (7)	5	2	0	Nền tảng tốt; thiếu cost/resource observability và interaction workflow hoàn chỉnh.
Tổng (39)	14	10	15	Tính đủ 39 component; V2-only và optional components vẫn được giữ để minh bạch scope.

3.7 Tổng hợp mapping

3.8 Những năng lực đã có và có giá trị để kế thừa

- **Plane và state boundary rõ:** control plane điều phối lifecycle; data plane phục vụ inference; execution plane dùng Docker hoặc Argo/Kubeflow; PostgreSQL và S3 giữ trạng thái bền vững.
- **Version immutability:** build thành công tạo model version với image URI/digest và artifacts; alias chuyển version hỗ trợ promotion/rollback mà không sửa version lịch sử.
- **Production event path:** gateway gửi event tenant/project/version-scoped sang Redpanda; consumer ghi vào PostgreSQL `paas_production_logs` và outbox tạo automatic-drift signal một cách transaction-safe.
- **Drift workflow:** DriftMonitor gắn version với reference asset; Evidently tạo report; DriftRun callback cập nhật theo UUID với idempotency key.
- **Training infrastructure:** TrainingJob hỗ trợ nhiều flavor/backend, resource constraints, capabilities-based security, và immutable code/data snapshots.

3.9 Các khoảng trống quan trọng cần diễn đạt chính xác

1. **Event contract không đầy đủ:** Consumer schema có các cột runtime và request correlation, nhưng event hiện chỉ publish identity tenant/project/version, timestamp, features và prediction. Confidence lấy từ worker được trả cho client nhưng **không đi vào event stream** tới production log.
2. **Prediction correlation chưa end-to-end:** Event id do model-server tạo (`uuid.uuid4()`) và được lưu vào production log, nhưng không trả về cho client dưới tên `prediction_id`. Client vì vậy chưa thể dùng ID này để submit ground-truth feedback sau đó.
3. **Evidently chỉ phân tích data drift:** Chưa đo F1, precision, recall trên nhãn production. Không có performance monitoring.

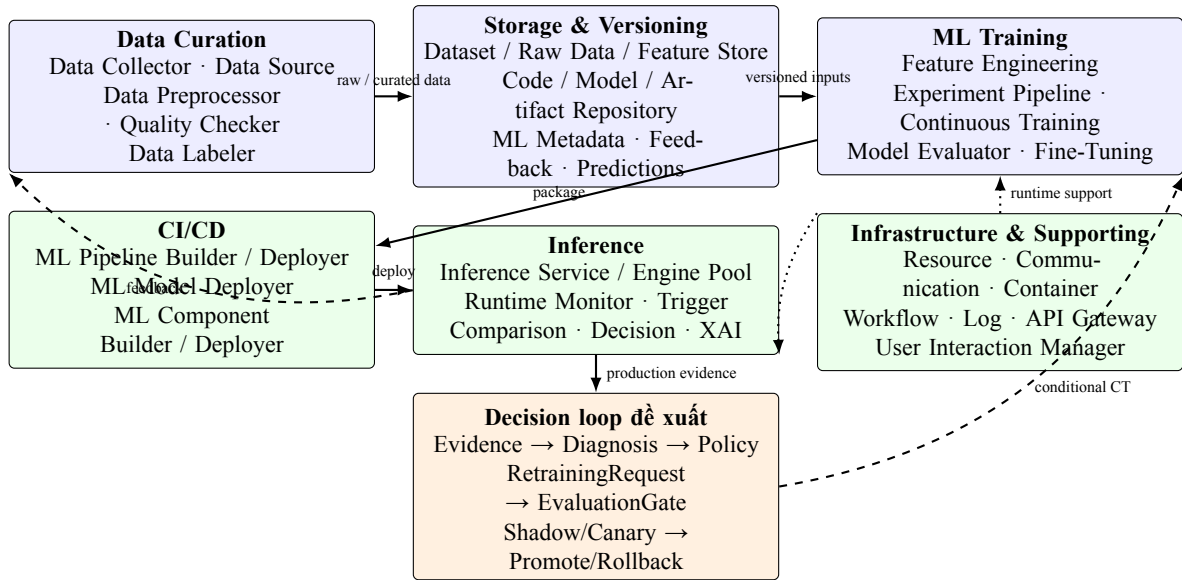


Figure 1: Bản đồ sáu nhóm component của paper và decision loop đề xuất cho repo. Màu xanh biểu diễn các nhóm nền tảng; màu cam là lớp cần xây dựng để chuyển từ V1 sang continuous training.

4. **Automatic drift outbox chỉ tạo DriftRun:** `request_automatic_drift_runs()` chỉ tạo DriftRun sau ngưỡng số bản ghi. Không có lời gọi trực tiếp tới training job, HPO hay promotion.
5. **Registry alias promotion/rollback là manual:** Frontend comparison trả về metrics diff rõ ràng và recommendation “unknown”, nên chưa phải champion–challenger evaluation.
6. **Không có traffic splitting:** Model-server gateway route 100% traffic tới một version duy nhất. Không có shadow/canary mechanism.

4 Đối chiếu quy trình lifecycle

Table 9: Trạng thái của các nhóm bước lifecycle then chốt trong framework 56 process steps [1].

Bước	Mức	Hiện trạng
Onboard, build, register	Có	Upload/training output → build image → immutable version.
Experiment/train	Một phần	Training job có snapshots và outputs; experimentation metadata phụ thuộc code người dùng/MLflow.
Deploy/serve	Có	Ready build được deploy; gateway xác thực và proxy theo tenant/project/version.
Collect production evidence	Một phần	Features/prediction được persist; cần hoàn chỉnh confidence/latency/status, prediction ID và feedback labels.
Detect degradation	Một phần	Data drift và infrastructure metrics có; prediction drift, quality, performance và slice metrics chưa chuẩn hoá.
Diagnose cause	Thiếu	Chưa có cause candidates, evidence graph, confidence hay policy explanation.
Select action and approve	Thiếu	Alert/run chưa thành RetrainingRequest; không có Auto/Human/Engineering routing.
Retrain / evaluate / promote	Một phần	Có train/build/register/deploy thủ công; thiếu candidate gate, shadow/canary, auto rollback và CT cooldown.

Điều này tạo một kết luận kiến trúc quan trọng: repo đã có *plumbing* cho feedback loop, nhưng chưa có *decision loop*. Bổ sung thêm một pipeline retraining đơn giản sẽ không giải quyết phần thiếu chính, vì data drift không đồng nghĩa model degradation và retrain lại data/code hiện hữu có thể lặp lại lỗi.

5 Hướng phát triển: Degradation-Aware Adaptive Model Maintenance

5.1 Luận đề và phạm vi

Hướng nghiên cứu được chọn là:

Hệ thống nên quyết định cần thay đổi gì trước khi retrain, dựa trên bằng chứng production, lineage và mức rủi ro; không trigger lại pipeline cũ chỉ vì một alert.

Vòng kín mục tiêu là:

Inference + feedback → evidence windows → degradation event → diagnosis → strategy/approval → CT/HPO → evaluation → shadow/canary → promote/rollback

Phiên bản nghiên cứu đầu tiên giới hạn vào ba action path có evidence rõ ràng:

1. **Harmful data drift:** distribution shift cùng performance degradation và labels đủ → Auto CT trong policy giới hạn.
2. **Data quality/schema/freshness incident:** → dừng CT, mở engineering incident; không retrain trên dữ liệu lỗi.
3. **Performance degradation chưa chắc nguyên nhân:** → Human-approved CT hoặc investigation, đặc biệt khi labels trễ/mâu thuẫn.

Code/feature semantics, label policy, architecture redesign và runtime outage nên là Engineering recommendation ở MVP. Configuration degradation/HPO là nhánh mở rộng sau khi ba path đầu đã có benchmark. Hình 2 biểu diễn thứ tự kiểm tra an toàn trước khi cho phép một candidate đi vào CT.

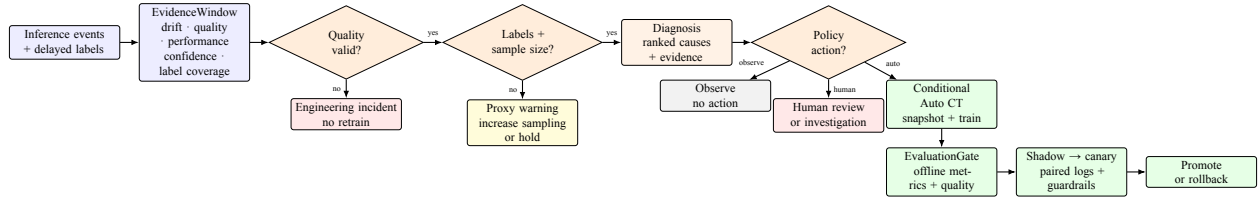


Figure 2: Workflow degradation-aware: quality gate và label sufficiency được kiểm tra trước diagnosis; diagnosis mới chọn observe, human review, engineering hoặc conditional Auto CT.

5.2 Câu hỏi nghiên cứu và giả thuyết

Để phân biệt đóng góp nghiên cứu với một backlog tính năng, evaluation sẽ trả lời ba câu hỏi:

1. **RQ1:** Evidence-to-action policy có giảm số lần retrain không cần thiết so với manual, periodic, alert-driven và always-retrain baselines mà vẫn duy trì recovery khi degradation có hại không?
2. **RQ2:** Diagnosis đa tín hiệu và delayed-label handling có giảm unsafe automation, tức tự động retrain/promotion trong scenario quality, mixed hoặc chưa đủ bằng chứng, không?
3. **RQ3:** Evaluation gate kết hợp shadow comparison có cải thiện promotion safety và chi phí vận hành so với retrain rồi deploy trực tiếp không?

Các giả thuyết tương ứng là: (H1) policy thích ứng giảm unnecessary CT trong benign drift; (H2) diagnosis + HITL giảm unsafe automation trong khi vẫn giữ được recovery ở harmful drift; (H3) candidate gate + shadow làm giảm false promotion mà không làm tăng time-to-recovery quá mức. Mỗi scenario cần được chạy với nhiều seed, báo cáo confidence interval và kiểm định paired trên cùng seed; các ngưỡng và ngân sách là biến thực nghiệm, không phải hằng số phổ quát.

5.3 Taxonomy evidence-to-action

Table 10: Một policy khởi đầu có thể giải thích được.

Evidence pattern	Diagnosis ưu tiên	Action an toàn
Data drift cao; quality ổn; labels đủ; performance giảm	Distribution shift / training data không còn đại diện	Snapshot data window mới; chỉ Auto CT nếu mọi policy, budget, sample-size, confidence-bound và evaluation gate đều pass.
Quality violation; schema, range hoặc freshness lỗi	Input/data pipeline issue	No CT; incident cho data/engineering owner.
F1/recall giảm; drift thấp hoặc mâu thuẫn	Unexplained degradation; concept/label/feature/implementation issue chưa phân biệt được	Human review, slice analysis, kiểm tra labels và lineage.
Confidence/entropy xấu; chưa có labels	Early warning hoặc calibration shift	Tăng sampling/monitoring; không Auto CT chỉ từ confidence.
Latency/error rate xấu	Serving/infrastructure issue	Rollback, autoscale hoặc sửa runtime; không CT.
Drift cao nhưng performance/risk ổn và đủ labels	Benign drift	Warning hoặc no action; tránh unnecessary retraining.

Confidence không phải thước đo universal của chất lượng model. Nó chỉ có giá trị làm evidence phụ khi được calibration theo model/version/segment; F1 chỉ có sau khi prediction được join với ground truth hợp lệ. Vì vậy prediction ID và delayed-label handling là **điều kiện tiên quyết**, không phải tiện ích phụ.

5.4 Diagnosis algorithm — Pseudocode

Rule-based cause scoring là lớp quyết định trung tâm. Dưới đây là pseudocode tối thiểu cho MVP diagnosis:

Trong pseudocode, quy ước $\Delta_{\text{perf}} = \text{current} - \text{baseline}$ cho metric càng cao càng tốt, nên $\theta_{\text{perf}} < 0$ biểu diễn suy giảm. Với metric càng thấp càng tốt, cần đảo chiều trước khi áp dụng. Các ngưỡng, minimum sample size và confidence level phải được cấu hình theo model/segment và được kiểm tra bằng sensitivity analysis.

Input: EvidenceWindow E với metrics $\{d_{\text{drift}}, q_{\text{quality}}, \Delta_{\text{perf}}, \Delta_{\text{conf}}, r_{\text{labels}}, n, CI_{\text{perf}}\}$

Step 1 (Data Quality Gate):

If $q_{\text{quality}} < \theta_{\text{quality}} \Rightarrow$ **Diagnosis:** DATA_QUALITY_ISSUE,

Action: ENGINEERING, **Confidence:** high

Step 2 (Harmful Drift):

If $d_{\text{drift}} > \theta_{\text{drift}}$ **and** $\Delta_{\text{perf}} < \theta_{\text{perf}}$ **and** $r_{\text{labels}} \geq r_{\text{min}}$ **and** $n \geq n_{\text{min}}$
and the lower bound of CI_{perf} confirms degradation,

$\Rightarrow$ **Diagnosis:** HARMFUL_DRIFT;

Action: AUTO_CT only if quality, cooldown, budget and candidate-gate preconditions pass; otherwise HUMAN_REVIEW;

Confidence: medium-high

Step 3 (Unexplained Performance Drop):

If $\Delta_{\text{perf}} < \theta_{\text{perf}}$ **and** $d_{\text{drift}} < \theta_{\text{drift}}$

$\Rightarrow$ **Diagnosis:** UNEXPLAINED_DEGRADATION with ranked candidates,

Action: HUMAN_REVIEW, **Confidence:** medium

Step 4 (Benign Drift):

If $d_{\text{drift}} > \theta_{\text{drift}}$ **and** $|\Delta_{\text{perf}}| \approx 0$ **and** $r_{\text{labels}} \geq r_{\text{min}}$

$\Rightarrow$ **Diagnosis:** BENIGN_DRIFT, **Action:** OBSERVE,

Confidence: medium; otherwise INSUFFICIENT_EVIDENCE

Step 5 (Confidence Early Warning):

If calibrated Δ_{conf} is significant **and** $r_{\text{labels}} < r_{\text{min}}$

⇒ **Diagnosis:** CALIBRATION_WARNING, **Action:** INCREASE_SAMPLING, **Confidence:** low

Default:

⇒ **Diagnosis:** AMBIGUOUS, **Action:** HUMAN_REVIEW, **Confidence:** low

Mỗi diagnosis bao gồm: (1) ranked cause candidates với confidence score, (2) evidence IDs tham chiếu, (3) explanation text cho audit trail. Khi hai causes có score gần nhau (ví dụ: data drift CAO + concept drift signals), hệ thống ưu tiên HUMAN_REVIEW để tránh unsafe automation.

5.5 Delayed-label handling strategy

Delayed labels là bottleneck then chốt nhất, vì không có labels thì performance metrics (F1, recall, precision) không tính được. Chiến lược xử lý:

1. **Prediction ID end-to-end:** Model-server trả `prediction_id` cho client trong response và giữ cùng correlation ID trong event. Client dùng ID này để submit ground-truth label qua Feedback API. Đây là **điều kiện tiên quyết** để join prediction-label.
2. **Join strategy — Watermark-based window:** Feedback store lưu labels với timestamp. Performance metrics tính trên window $[t - W, t]$ khi label coverage $r_{\text{labels}} = \frac{|\text{labeled predictions}|}{|\text{total predictions}|} \geq r_{\text{min}}$. Giá trị r_{min} (ví dụ 0.1, 0.3 hoặc 0.5) là tham số cần sensitivity analysis, không phải khuyến nghị phổ quát. Nếu coverage thấp, hệ thống ghi nhận “insufficient labels” và **không** tính confirmed performance.
3. **Proxy metrics khi labels trễ:** Confidence distribution shift, prediction distribution shift và latency anomalies là proxy signals. Chúng **không** đủ để kết luận degradation nhưng đủ để tăng monitoring sampling hoặc tạm hold deployment.
4. **Label quality/trust:** Mỗi label record có `source` (automated pipeline, human annotator, implicit feedback) và `trust_level`. Không nên âm thầm trộn các nguồn nhãn khác chất lượng; cần dùng trust để quyết định eligibility hoặc báo cáo metrics phân tầng, thay vì mặc định thay đổi trọng số F1.
5. **Partial label availability:** Khi chỉ có một phần predictions được label (ví dụ 30%), performance metric tính trên labeled sample với confidence interval và kiểm tra selection bias. Decision engine chỉ trigger Auto CT khi lower bound confidence interval vẫn cho thấy degradation.

5.6 State model tối thiểu

Table 11: Các entity cần bổ sung vào control plane.

Entity	Trường cốt lõi	Mục đích
PredictionRecord	<code>prediction_id</code> , <code>request_id</code> , <code>tenant/project/version</code> , <code>timestamp</code> , <code>prediction</code> , <code>confidence</code> , <code>latency</code> , <code>status</code>	Ghi nhận một lần inference và tạo correlation key ổn định cho feedback.
Feedback	<code>feedback_id</code> , <code>prediction_id</code> , <code>ground_truth</code> , <code>label_timestamp</code> , <code>label_source</code> , <code>trust_level</code> , <code>correction_of</code>	Nối label trễ với prediction; hỗ trợ nhiều feedback/correction và provenance.

Entity	Trường cột lỗi	Mục đích
EvidenceWindow	model_version, window_start/end, baseline_ref, metric_values (drift, quality, perf, confidence), slices, label_coverage, policy_version	Lưu bằng chứng bất biến theo cửa sổ thay vì chỉ report file.
DegradationEvent	evidence_window_ids, severity, affected_component, deduplication_key, status, created_at	Chuẩn hoá alert có thể idempotent và audit được.
Diagnosis	degradation_event_id, ranked_cause_candidates (JSON), top_confidence, explanation, lineage_snapshot, policy_version	Tách suy luận nguyên nhân khỏi detection.
RetrainingRequest	diagnosis_id, strategy (auto/human/engineering), data/code/config_version, budget_limit, cooldown_until, approval_status, approved_by	Là đơn vị điều phối thay cho “DriftRun gọi TrainingJob”.
EvaluationGate	candidate_version, champion_version, offline_metrics, calibration_score, latency_p95, error_rate, data_quality_checks, business_checks, decision (pass/fail/hold), decided_at	Chặn candidate không đủ bằng chứng trước production.
TrafficExperiment	experiment_type (shadow/canary), candidate_version, champion_version, cohort_hash_seed, weight_schedule (JSON), guardrail_thresholds, current_weight, status, rollback_reason	Ghi lại so sánh production và rollback path.

Một prediction có thể chưa có label hoặc có nhiều lần label/correction; vì vậy PredictionRecord và Feedback nên là hai bảng có quan hệ một-nhiều, kèm unique/idempotency constraint cho nguồn feedback và version correction.

6 Ranh giới tự động hoá và promotion

Table 12: Ranh giới Auto–Human–Engineering cho MVP.

Mức	Hệ thống được phép làm	Không tự làm
Observe / no action	Tạo event, tăng sampling, theo dõi window tiếp theo.	Không tạo training storm chỉ từ warning.
Auto CT	Chọn data window đã phê duyệt, snapshot, train, evaluate, register candidate.	Không đổi label definition, feature semantics, source code hay architecture.
Human-approved CT	Chuẩn bị recommendation, lineage và candidate plan.	Không tự quyết risk/business trade-off hoặc data adequacy mơ hồ.
Engineering intervention	Tạo issue/audit evidence và chờ Git/PR/CI.	Không tự sửa pipeline, dependency, feature code hoặc runtime.

6.1 Shadow/Canary architecture

Promotion trong kiến trúc đề xuất nên dùng **shadow + canary**; đây là capability tương lai, chưa phải implementation hiện tại:

1. **Shadow mode (Phase 5a):** Candidate nhận traffic duplicate tại gateway level nhưng response không ảnh hưởng user. Kiến trúc đề xuất là model-server gateway fork request tới cả champion worker và candidate worker; chỉ trả response từ champion. Candidate predictions được log riêng vào `paas_shadow_logs` để so sánh offline trên cùng request/correlation ID.
2. **Canary mode (Phase 5b):** Weighted routing thực sự – model-server gateway dùng consistent hashing trên một cohort key ổn định (user/session/request ID), không chỉ `tenant_id`, để route traffic. Lộ trình tăng weight 5% → 10% → 25% → 50% → 100% chỉ là lịch thử nghiệm; mỗi bước chỉ tăng khi guardrails đạt.
3. **Cohort consistency:** Dùng hash-based routing đảm bảo cùng user/session luôn đi vào cùng version trong một canary phase, tránh experience inconsistency. Nếu chọn canary theo tenant, phải ghi rõ đó là tenant-level rollout và không diễn giải là 5% request-level traffic.
4. **Guardrail evaluation:** Đánh giá theo cửa sổ cấu hình được, với minimum sample size, confidence interval và paired champion–candidate logs. Ví dụ guardrails gồm:
 - Error rate candidate $\leq$ champion + ϵ
 - Latency p95 candidate $\leq$ champion $\times$ 1.2
 - Prediction distribution KL-divergence $\leq$ threshold
 - Nếu labels available: performance metric candidate $\geq$ champion $-\delta$
5. **Rollback:** Rollback tự động khi guardrail nghiêm trọng hoặc vi phạm trong k cửa sổ liên tiếp (có hysteresis/cooldown), thay vì phản ứng với một mẫu đơn lẻ. Ghi `rollback_reason` vào `TrafficExperiment`. Blue–green alias swap chỉ là fast recovery khi cả hai endpoint đã healthy và routing/alias hỗ trợ atomic switch; hiện đây vẫn là capability đề xuất.

Lưu ý: Shadow mode đơn giản hơn nhiều so với canary và đủ cho paper nếu timeline tight. Canary là stretch goal. Hình 3 cho thấy promotion là một chuỗi gate, không phải alias swap ngay sau training.

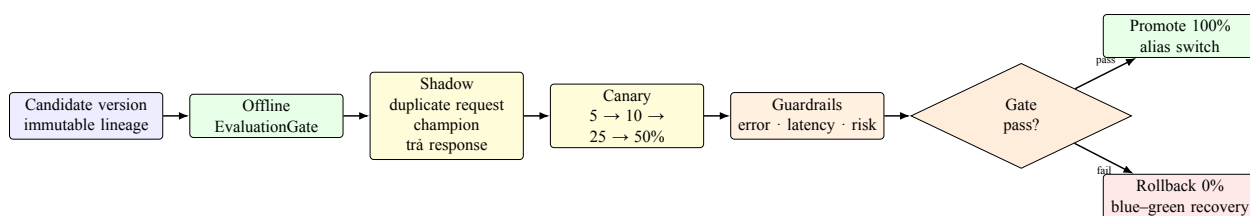


Figure 3: Protocol champion–challenger đề xuất: candidate phải qua offline gate, shadow và guardrails trước khi promote; rollback là nhánh an toàn khi gate thất bại.

7 Lộ trình hiện thực và nghiên cứu

Table 13: Lộ trình đề xuất 26 tuần, gồm 24 tuần phát triển và 2 tuần viết paper song song.

Mốc	Mục tiêu	Deliverables	Tiêu chí hoàn thành
— Giai đoạn chuẩn bị —			
Phase 0 (Tuần 1–2)	Freeze protocol nghiên cứu	Dataset/task, scenario injection framework, ground-truth causes, baselines, budget, metrics, seeds. Chọn 1–2 task tabular classification/regression.	Benchmark protocol được version-control (Git + immutable S3 manifest; DVC là optional). Document chọn dataset, scenarios, metrics trước khi có kết quả.
Phase 0.5 (Tuần 3–4)	Vertical Slice PoC	End-to-end trên 1 scenario (S2 harmful drift): inject drift → detect → diagnose (hardcoded) → retrain → evaluate offline → register candidate → compare manual.	Pipeline integration validated; các blocking issues được phát hiện sớm.
— Giai đoạn xây dựng —			
Phase 1 (Tuần 5–9)	Measurement foundation	1a (Tuần 5–6): Prediction ID end-to-end, confidence/latency/status event contract, sửa <code>send_to_redpanda()</code> gửi đầy đủ fields. 1b (Tuần 7–8): Feedback API + store, PredictionRecord entity, join prediction–label integration test. 1c (Tuần 9): Immutable data snapshot, EvidenceWindow entity, data quality checks cơ bản.	Join được prediction–label trên synthetic data. Truy được model → training → data/code/config. Consumer nhận và persist đầy đủ event fields.

Mốc	Mục tiêu	Deliverables	Tiêu chí hoàn thành
Phase 2 (Tuần 10–13)	Detection	2a (Tuần 10–11): Mở rộng Evidently: data drift, prediction distribution drift, quality violations (schema, range, freshness), baseline/threshold/slice. 2b (Tuần 12): Performance monitoring với delayed labels: tính F1/recall/precision khi label coverage $\geq r_{\min}$. 2c (Tuần 13): DegradationEvent entity, idempotent event creation, test coverage scenarios S0–S3.	Stable (S0), benign drift (S1), harmful drift (S2), quality (S3) tạo đúng event type, idempotent. Detection precision/recall đo được trên synthetic scenarios.
Phase 3 (Tuần 14–17)	Diagnosis + HITL	3a (Tuần 14–15): Diagnosis entity, rule-based cause scoring (pseudocode Section 5.4), explanation text, evidence IDs. 3b (Tuần 16): RetrainingRequest entity, cooldown/budget policy, approval API. 3c (Tuần 17): Approval UI/inbox, audit trail, integration tests: warning $\nrightarrow$ train; mỗi decision có evidence IDs; conflict resolution cho multi-cause.	Unit/integration tests chứng minh warning không gọi train trực tiếp. Diagnosis top-k/multi-label accuracy đo được trên scenarios S0–S4, S9.
— Giai đoạn CT pipeline —			
Phase 4 (Tuần 18–20)	CT candidate	Dataset builder (data window mới + validated), constrained HPO (nếu có), train, EvaluationGate entity, registry candidate với champion lineage.	Candidate có reproducible lineage và bị reject/promote theo policy. Offline evaluation gate pass/fail/hold hoạt động.

Mốc	Mục tiêu	Deliverables	Tiêu chí hoàn thành
Phase 5 (Tuần 21–23)	Production comparison	5a (Tuần 21–22): Shadow routing ở gateway level, shadow logs, offline comparison metrics. 5b (Tuần 23, stretch): Canary weighted routing, guardrails, automatic rollback, alias promotion.	MVP có champion–challenger experiment với shadow comparison. Canary là stretch; nếu chưa hoàn thành thì không claim production promotion safety.
— Giai đoạn đánh giá —			
Phase 6 (Tuần 24–26)	Benchmark / paper	Run baselines, ablation, sensitivity analysis. Cost/human-effort analysis. Artifact release. Paper viết và review nội bộ.	Kết quả lặp lại được với bảng, plots, limitations và raw metadata.

Lưu ý về timeline: So với bản lộ trình 20 tuần ban đầu, phiên bản này bổ sung: (1) Phase 0.5 PoC để validate integration sớm, (2) mở rộng Phase 1 thành 5 tuần (từ 4) vì event contract refactor cần sửa cả model-server, consumer và control-plane, (3) mở rộng Phase 3 thành 4 tuần (từ 3) vì diagnosis + approval UI là module mới hoàn toàn, (4) tách Phase 5 thành shadow (core) và canary (stretch). Tổng timeline là 26 tuần, trong đó 24 tuần phát triển và 2 tuần cuối dành cho benchmark/paper song song.

7.1 Backlog theo ownership hiện tại

- **model-server + consumer:** Hoàn chỉnh event contract để gửi prediction ID, confidence, latency và status. Consumer persist đầy đủ fields. Bảo đảm durable hand-off để feedback join được event.
- **control plane:** Thêm Feedback API, EvidenceWindow, DegradationEvent, Diagnosis, RetrainingRequest, EvaluationGate, TrafficExperiment entities. Giữ UUID, tenant scope, callback idempotency và PostgreSQL/S3 state ownership.
- **evidently service:** Mở rộng từ data drift → prediction distribution drift, quality checks, performance metrics (khi labels available).
- **training-runner + execution backends:** Nhận immutable snapshot, dataset window/config policy và xuất standardized metrics/artifacts.
- **registry + deployment + GitOps:** Candidate/champion metadata, TrafficExperiment, weighted routing/rollout và rollback guardrails.
- **web frontend:** Evidence timeline, approval inbox, candidate comparison thật, shadow/canary dashboard và audit trail.

8 Thiết kế thực nghiệm

Để claim nghiên cứu có giá trị, benchmark phải biết *ground-truth cause* của từng scenario, không chỉ ground-truth label của bài toán ML.

Table 14: Benchmark scenarios phân theo priority.

ID	Priority	Scenario	Expected action
S0	Core	Stable (no degradation)	No action
S1	Core	Benign data drift (performance ổn)	Warning, observe
S2	Core	Harmful data drift (performance giảm, labels đủ)	Auto CT nếu các gate pass
S3	Core	Data quality violation (schema, range, freshness)	Engineering, no CT
S4	Important	Concept/label change	Human-approved CT
S9	Important	Mixed/ambiguous (nhiều cause)	Human review
S5	Stretch	Training-data aging (gradual)	Auto CT hoặc human
S6	Stretch	Configuration degradation	HPO/config update
S7	Stretch	Code/feature issue	Engineering
S8	Stretch	Runtime / infrastructure issue (confounder, không phải model drift)	Rollback/autoscale

8.1 Benchmark scenarios — chia priority

Khởi đầu nên dùng một hoặc hai task tabular classification/regression để kiểm soát chi phí và tái lập. Mỗi scenario có seed, thời điểm inject, cường độ, affected component, expected diagnosis và expected action; tách timeline baseline, injection, detection, intervention và recovery.

8.2 Baselines

- **B0 – Observe-only:** Không retrain, dùng để đo degradation và recovery tự nhiên.
- **B1 – Manual retraining:** Người dùng tự quyết định retrain.
- **B2 – Periodic CT:** Retrain theo lịch cố định (mỗi tuần/tháng).
- **B3 – Alert-driven CT:** Retrain mỗi khi có drift alert; đây là baseline đối chứng, chưa phải luồng hiện tại của repo.
- **B4 – Always retrain:** Stress/cost baseline, retrain sau mỗi batch data.
- **B5 – Proposed adaptive CT:** Detection → Diagnosis → Policy → Conditional CT.

Các baseline B0–B3 và B5 nên có budget compute tương đương; B4 là stress/cost baseline. Với B1, protocol thao tác của người dùng, thời gian phản hồi và tiêu chí quyết định phải được cố định để bảo đảm tái lập.

8.3 Evaluation metrics

Không nên chỉ báo cáo F1 cuối cùng: recovery theo thời gian và chi phí của false retraining mới kiểm chứng được policy có ích.

Table 15: Evaluation metrics theo dimension.

Dimension	Metrics
Detection quality	Precision, recall, F1 of detection; detection latency (time from injection to first alert); data-quality detection precision/recall
Diagnosis quality	Top-k and multi-label accuracy (is ground-truth cause in top-k?); confidence calibration (ECE/Brier)
Strategy quality	Strategy accuracy (correct action for cause); unsafe automation rate; false promotion rate
Recovery	Time-to-recovery; performance recovery curve; slice-level recovery
Cost efficiency	Unnecessary CT count; compute hours per strategy; storage cost; serving cost for shadow/canary
Human effort	Approval burden; review time; override rate; false alarm rate reported separately from labor
Promotion safety	Guardrail violation rate; rollback rate/success; minimum-sample compliance; candidate/champion confidence interval
Reproducibility	Lineage completeness; scenario replay success; rollback success rate
Statistical validity	Multiple seeds, confidence intervals and paired tests across the same scenario/seed

8.4 Cost model

- Mỗi CT cycle có chi phí: $C_{CT} = C_{data_prep} + C_{train} + C_{evaluate} + C_{shadow/canary}$; báo cáo riêng CPU-hours/GPU-hours, storage và human-hours rồi mới quy đổi sang đơn giá cụ thể.
- Budget policy: tối đa N CT runs/tuần, tổng compute hours/tháng $\leq B$
- Cost-benefit: CT chỉ justified khi $expected_performance_gain \times business_value > C_{CT} + C_{risk}$
- Metric so sánh strategies: **performance gained per compute dollar** (efficiency ratio), với giả định business value được công bố rõ hoặc thay bằng proxy performance thống nhất giữa các scenario.

9 Rủi ro và nguyên tắc kiểm soát

- **Delayed labels:** Performance metric đến muộn. Thiết kế phải phân tách proxy monitoring với confirmed degradation; lưu timestamp và source của label. Chi tiết chiến lược xem Section 5.5.
- **Causal ambiguity:** Nhiều cause cùng lúc. Diagnosis trả ranked candidates, evidence coverage và no-action/human fallback; không tuyên bố root cause tuyệt đối ngoài controlled scenarios.
- **Unsafe automation:** Dùng policy-as-code, budget/concurrency/cooldown, immutable lineage, approval boundary và rollback availability.
- **Multi-tenancy/privacy:** Raw feature/prediction/feedback có thể nhạy cảm. Feedback store và evidence query phải tenant-scoped, retention-aware và tối thiểu hoá dữ liệu lưu.
- **External validity:** Synthetic drift đơn giản có thể làm kết quả quá lạc quan. Cần varied intensity, missing/delayed labels, mixed causes và temporal replay trước khi đưa kết luận rộng.
- **Training storm:** Cooldown period tối thiểu giữa các CT runs cho cùng model version. Budget cap ngăn cascade retraining.

10 Related Work

10.1 MLOps architecture và reference frameworks

Amou Najafabadi et al. [1] cung cấp systematic mapping study toàn diện nhất cho MLOps architecture. Google [3] đề xuất ba mức maturity MLOps (ML0–ML2) với CT ở ML2. Microsoft [4] mô tả maturity model năm mức. Bayram và Ahmed [5] survey trustworthy ML in production với focus vào robustness.

10.2 Model monitoring và drift detection

Evidently (open-source) cung cấp data drift, prediction drift và data quality monitoring [6]. NannyML tập trung vào performance estimation khi labels chưa có [7]. Các phương pháp drift detection học thuật bao gồm ADWIN [8], Page-Hinkley test, và DDM [9]. Lu et al. [10] cung cấp survey toàn diện về concept drift.

10.3 Continuous training và adaptive retraining

Luo et al. [11] đề xuất open-source pipeline cho autonomously adaptive ML. Semola et al. [12] giới thiệu CLaaS (Continual Learning as a Service). Seldon Core và BentoML cung cấp model serving với canary deployment capabilities. Trong phạm vi tài liệu được khảo sát ở đây, nhiều công trình tập trung vào *khi nào* retrain (trigger); mức độ hỗ trợ cho *tại sao* và *nên làm gì* (diagnosis → action selection) cần được kiểm chứng bằng một systematic comparison riêng. Đây là gap nghiên cứu được đề xuất, không phải kết luận phổ quát từ các tài liệu hiện có.

11 Kết luận

Repository đã đủ các mảnh ghép để trở thành experimental platform tốt cho nghiên cứu MLOps: immutable model lifecycle, execution infrastructure, serving, production event ingestion và data drift. Mapping đủ 39 thành phần cho thấy 14 đã có, 10 một phần và 15 thiếu theo tiêu chí heuristic của report — trong đó các khoảng trống quan trọng nhất là Feedback Database, Continuous Training Pipeline, Model Comparison Runner, Data Quality Checker và Trigger → CT connection.

Giá trị nghiên cứu tiếp theo không nằm ở việc thêm “auto retrain sau drift”, mà ở lớp **decision intelligence** có thể giải thích được giữa monitoring và execution. Lộ trình 26 tuần với 7 phase (bao gồm Phase 0.5 PoC và buffer; 24 tuần phát triển và 2 tuần paper song song) cung cấp con đường khả thi để repo hỗ trợ một đóng góp rõ ràng về degradation-aware adaptive model maintenance: duy trì recovery tương đương hoặc tốt hơn trong khi giảm unnecessary retraining, chi phí và unsafe automation.

Ba deliverable chính cho paper:

1. **Taxonomy:** Evidence-to-action mapping (Table 10) với diagnosis algorithm có thể giải thích và đánh giá (Section 5.4).
2. **Platform:** Open-source implementation trên MLOps PaaS repo hiện có, chứng minh tính khả thi của approach.
3. **Benchmark:** Evaluation trên synthetic scenarios với ground-truth causes, so sánh adaptive CT với base-lines theo cả performance, cost và human effort.

References

- [1] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, and P. Lago, “An architectural perspective on MLOps: Structures, processes, tools, and stakeholders,” *Information and Software Technology*, vol. 193, Art. 108029, 2026. doi: [10.1016/j.infsof.2026.108029](https://doi.org/10.1016/j.infsof.2026.108029).
- [2] MLOps PaaS repository, source inspection of control plane, model server, consumer, Evidently, training, registry, deployment, GitOps manifests, and architecture documentation, accessed August 28, 2026.
- [3] K. Salama, J. Kazmierczak, and D. Schut, “Practitioners guide to MLOps: A framework for continuous delivery and automation of machine learning,” Google Cloud, 2021.
- [4] Microsoft, “MLOps maturity model,” Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/mlops-maturity-model>.

- [5] F. Bayram and B. S. Ahmed, “Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach,” *ACM Computing Surveys*, vol. 57, no. 5, pp. 1–35, 2025.
- [6] Evidently AI, “Evidently: Open-source ML monitoring,” <https://www.evidentlyai.com/>.
- [7] NannyML, “NannyML: Performance estimation and monitoring,” <https://www.nannyml.com/>.
- [8] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptive windowing,” in *Proc. SDM*, SIAM, 2007, pp. 443–448.
- [9] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, “Learning with drift detection,” in *SBIA*, Springer, 2004, pp. 286–295.
- [10] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, 2019.
- [11] Y. Luo, M. Raatikainen, and J. K. Nurminen, “Autonomously adaptive machine learning systems: Experimentation-driven open-source pipeline,” 2023.
- [12] R. Semola, V. Lomonaco, and D. Bacciu, “Continual-learning-as-a-service (CLaaS): On-demand efficient adaptation of predictive models,” 2023.